

COS30082

Applied Machine Learning



Lecture 3

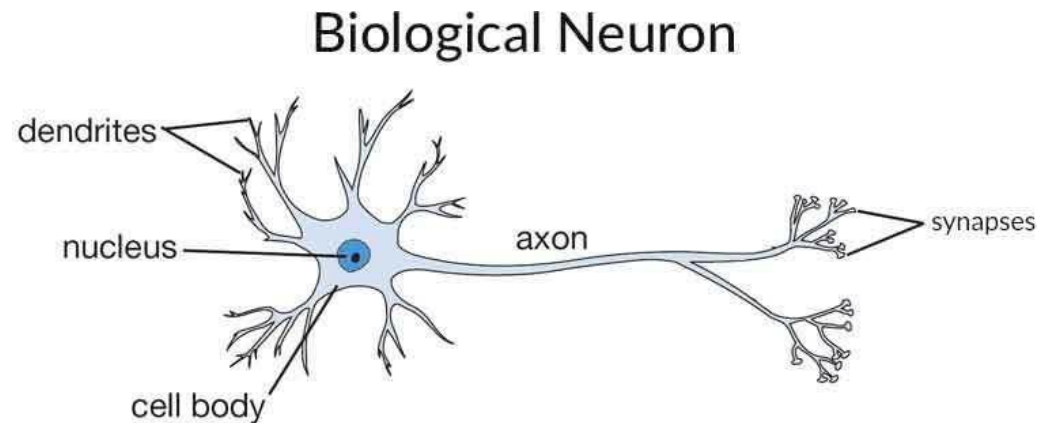
Deep Neural Networks

Why Neural Networks?

- Capture complex non-linear relationships in data
- Automatically learn useful features from raw inputs
- Work well with high-dimensional data (images, text, audio)
- Often achieve higher prediction accuracy on complex tasks
-

Understanding of Artificial Neural Network (ANN)

- The human brain contains hundreds of billions of cells called **neurons**. Each neuron has a cell body that processes information by receiving signals from other neurons (inputs) and sending signals to other neurons (outputs).

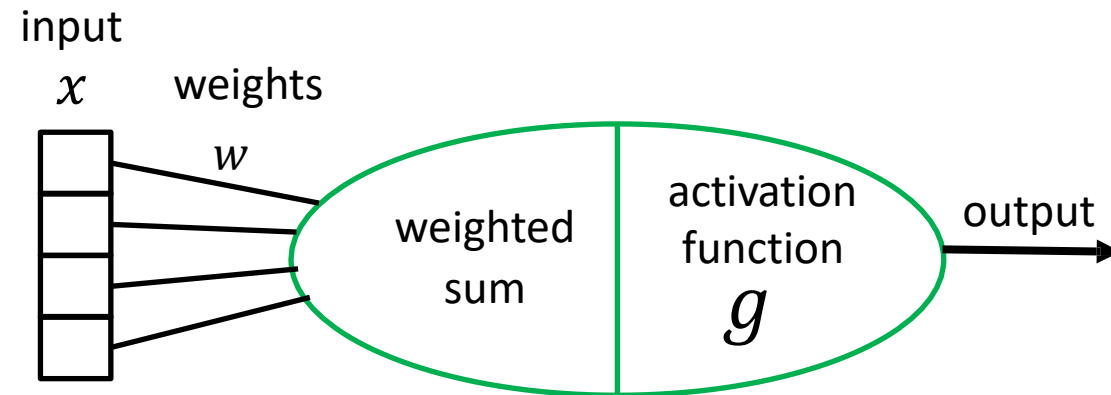
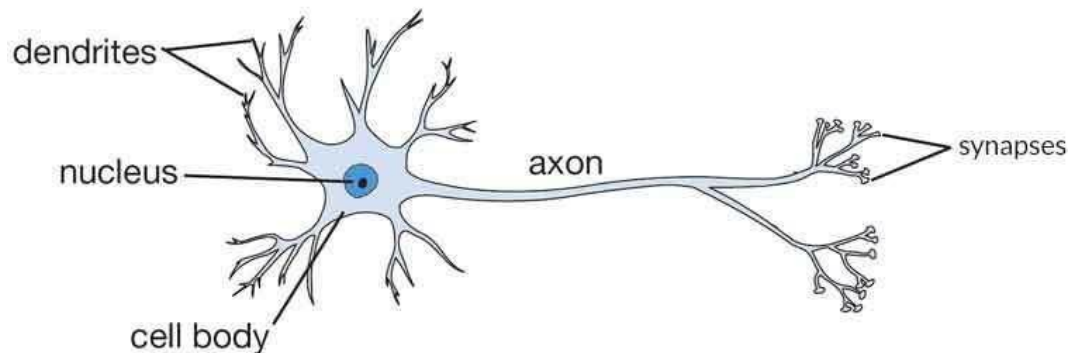


<https://www.xenonstack.com/blog/artificial-neural-network-applications/>

Understanding of ANN

- Artificial neural networks are built like the human brain. It has hundreds or thousands of artificial neurons which are interconnected. The input neurons receive various forms of data, and based on an internal weighting system, the neural network learns the patterns of the data and matches them to the target output.

Biological Neuron



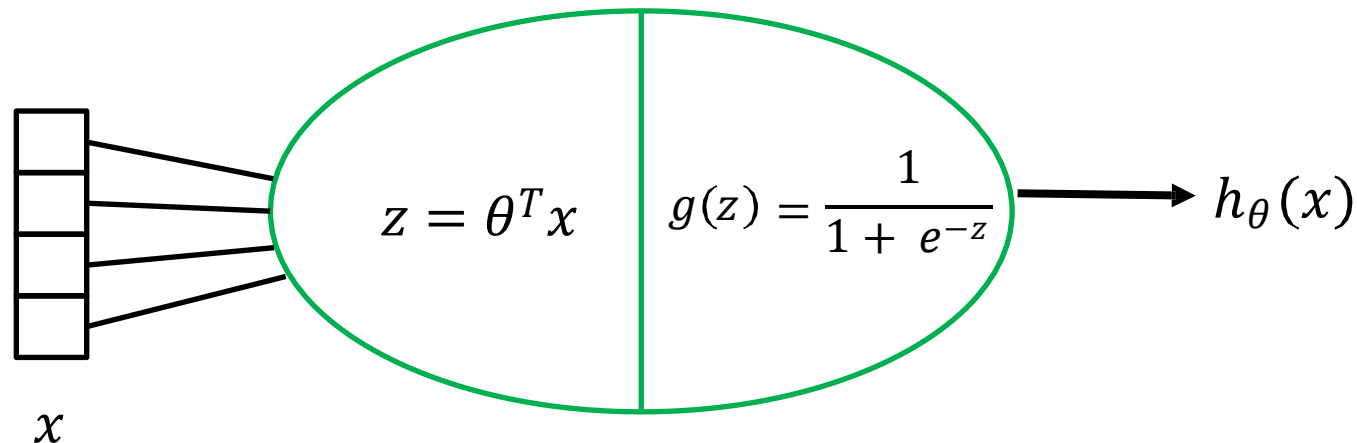
Neuron model for logistic regression

- In binary classification, to ensure predicted value $h_{\theta}(x)$ lies between 0 and 1:

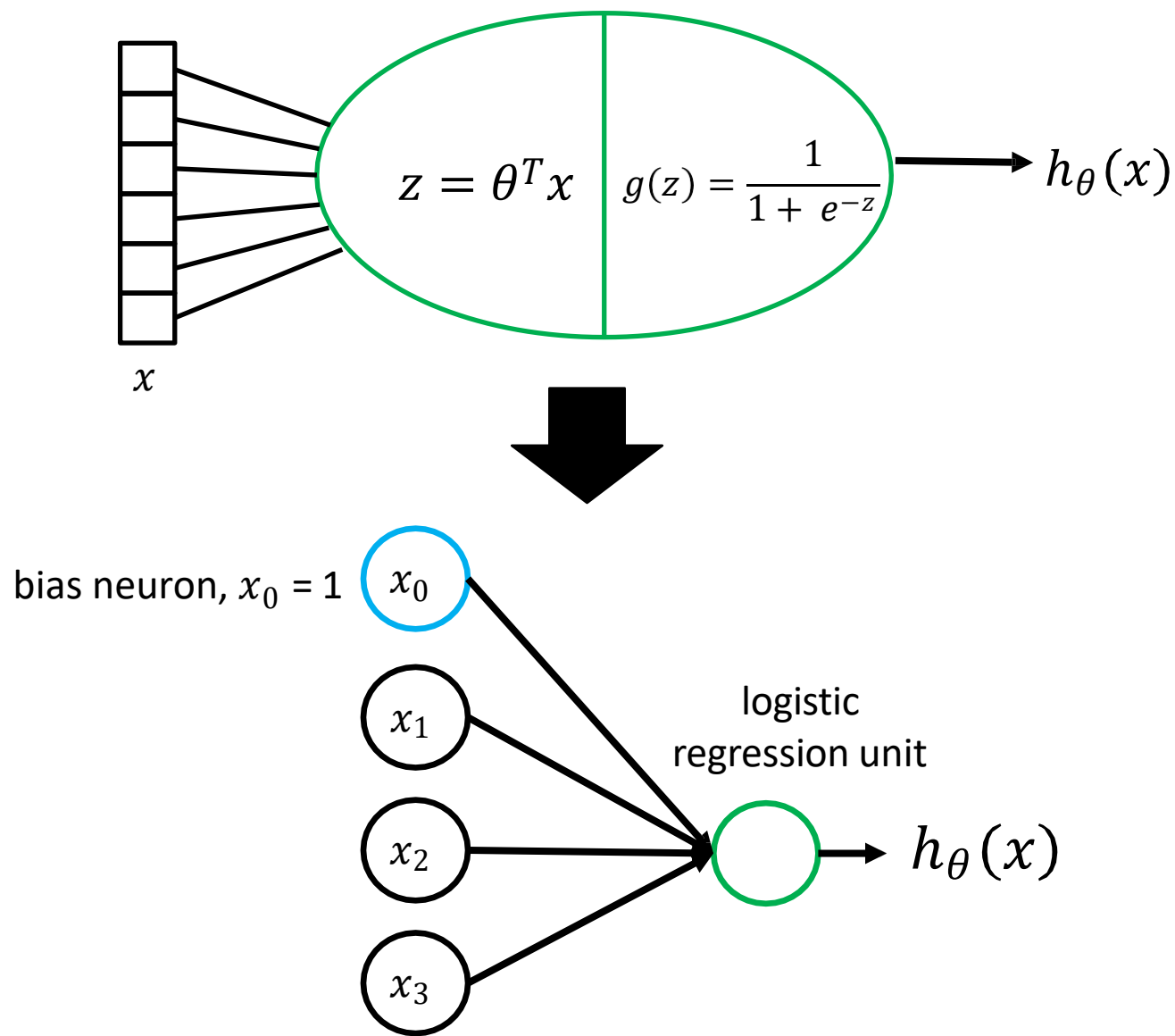
$$0 \leq h_{\theta}(x) \leq 1$$

$h_{\theta}(x)$ is represented as $\rightarrow h_{\theta}(x) = g(z)$

where $z = \theta^T x$ $g(z) = \frac{1}{1 + e^{-z}}$



Neuron model for logistic regression



- Says an input x with 3 dimensional features
- The neuron model terminologies for LR:

$$x = \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

input

$$\theta = \begin{bmatrix} \theta_0 \\ \theta_1 \\ \theta_2 \\ \theta_3 \end{bmatrix}$$

weights / parameters

$$g(z)$$

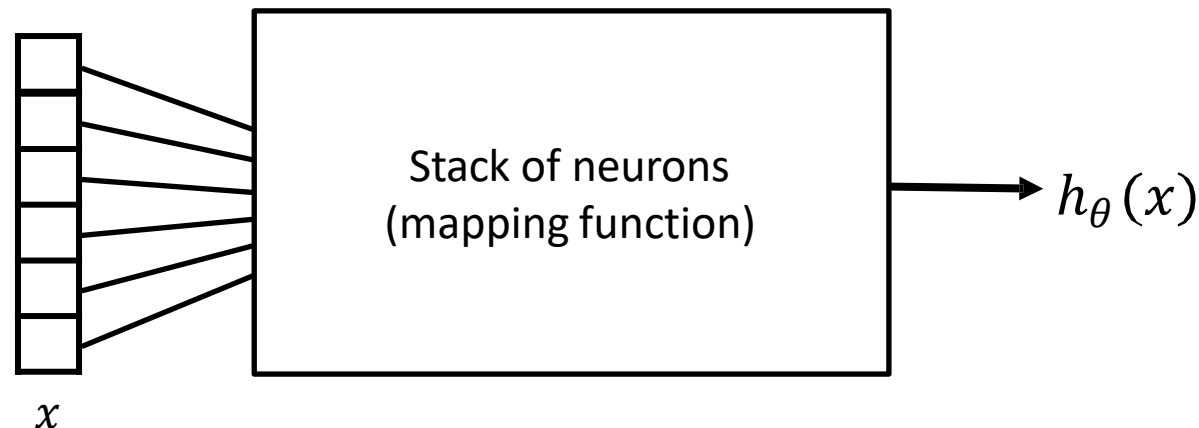
sigmoid (regression)
activation function

$$h_{\theta}(x)$$

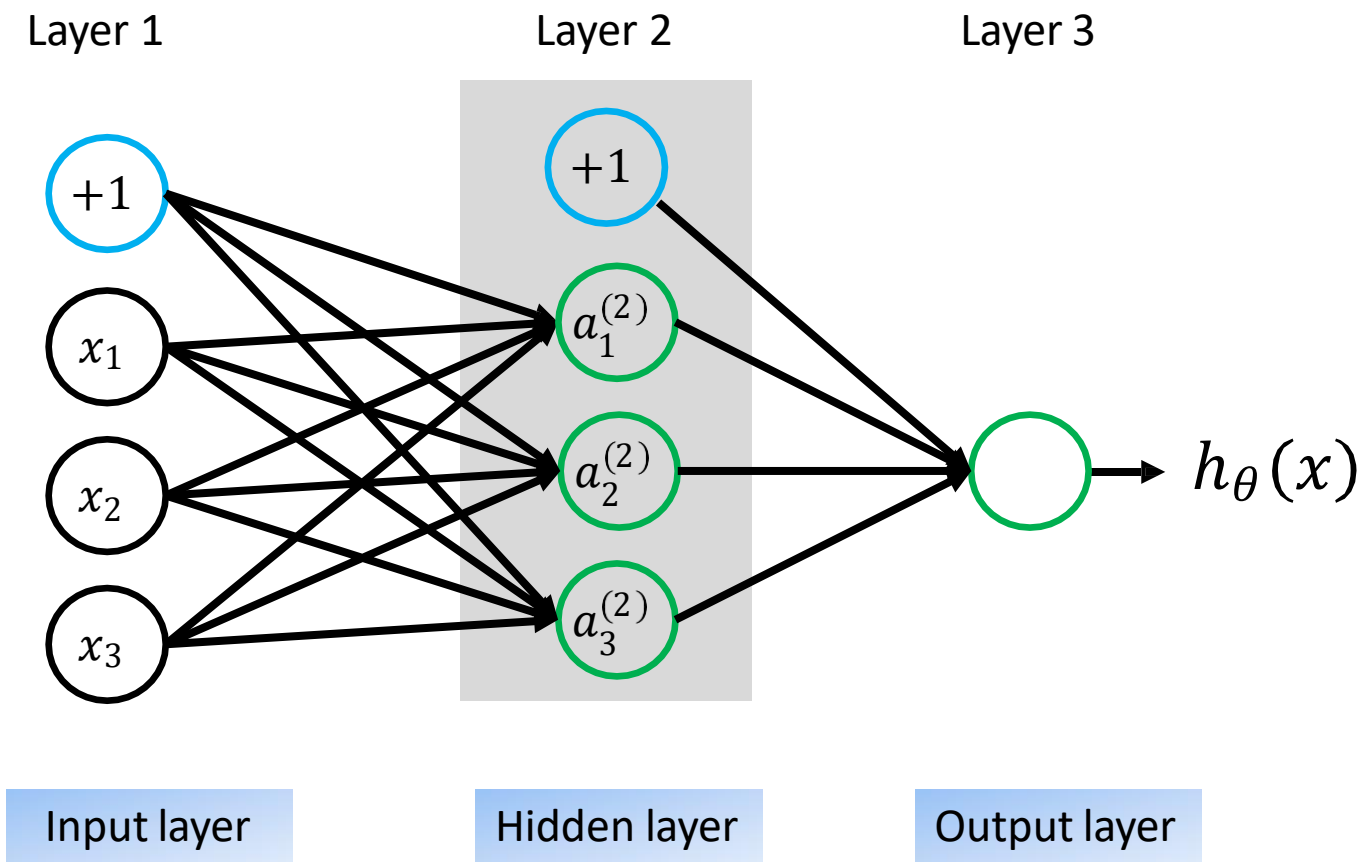
predicted output

Neural network

- Neural network consists of stack of neurons that takes in input x and output predicted value $h_{\theta}(x)$
- Given enough number of neurons, neural network are incredibly good at mapping input x to the actual output y .

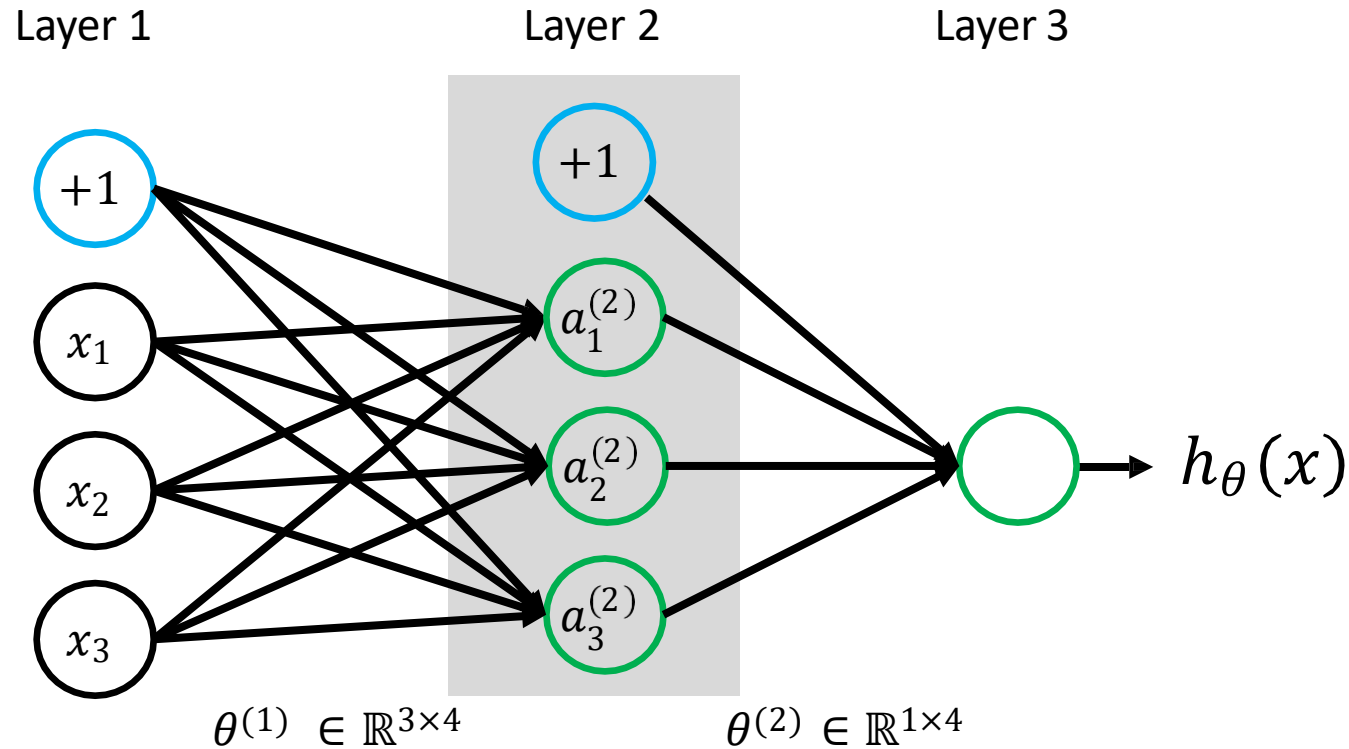


Neural network (single hidden layer)



- Notation used in computation:
 - $a_i^{(j)}$: activation of neuron i in layer j
 - θ^j : matrix of weights make up the mapping function from layer j to $j + 1$
 - s_j : number of neurons in layer j not including bias

Neural network (single hidden layer)



- Notation used in computation:

$a_i^{(j)}$: activation of neuron i in layer j

θ^j : matrix of weights make up the mapping function from layer j to $j + 1$

s_j : number of neurons in layer j not including bias

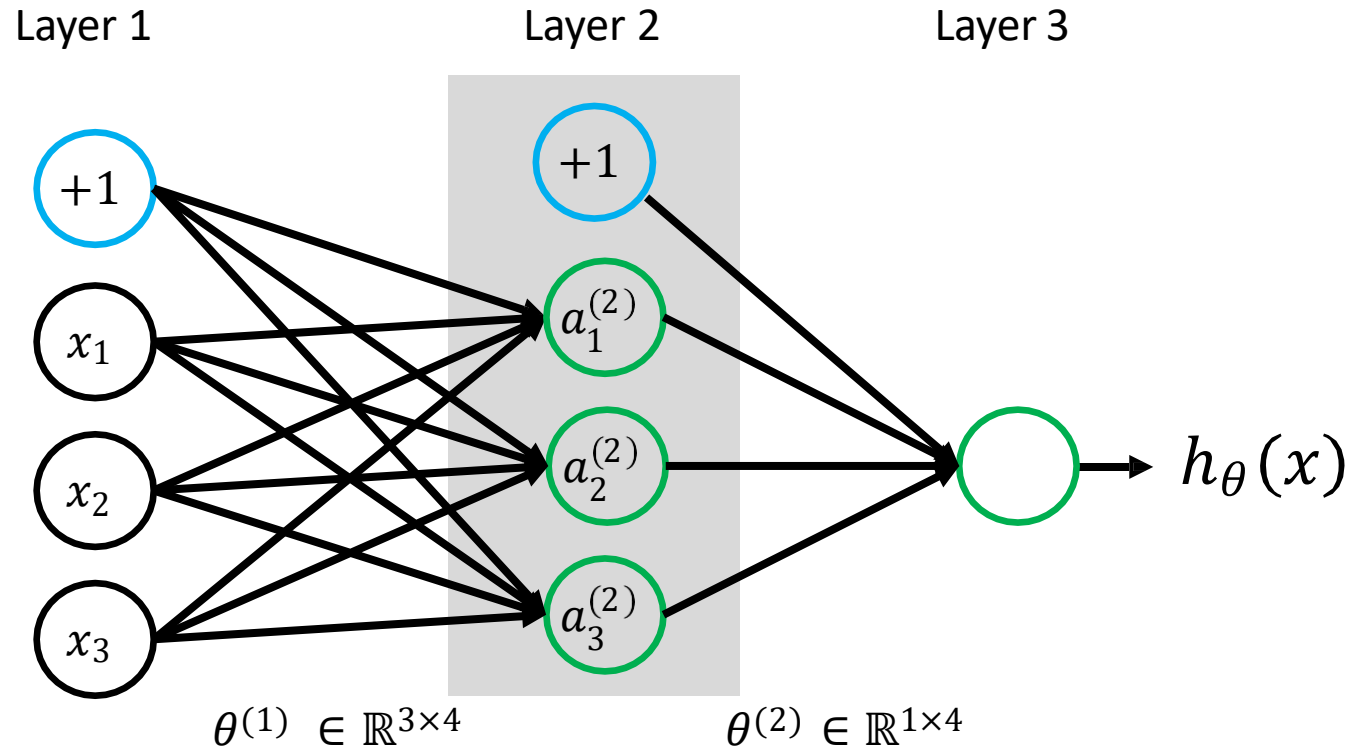
$$\begin{aligned}
 a_1^{(2)} &= g(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3) \\
 a_2^{(2)} &= g(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3) \\
 a_3^{(2)} &= g(\theta_{30}^{(1)} x_0 + \theta_{31}^{(1)} x_1 + \theta_{32}^{(1)} x_2 + \theta_{33}^{(1)} x_3) \\
 h_{\theta}(x) &= a_1^{(3)} = g(\theta_{10}^{(2)} a_1^{(2)} + \theta_{11}^{(2)} a_2^{(2)} + \theta_{12}^{(2)} a_3^{(2)} + \theta_{13}^{(2)} a_4^{(2)})
 \end{aligned}$$

Matrix format:

$$z^{[l]} = W^{[l]} a^{[l-1]} + b^{[l]}$$

$$a^{[l]} = g^{[l]}(z^{[l]})$$

Neural network (single hidden layer)



$$a_1^{(2)} = g(\theta_{10}^{(1)} x_0 + \theta_{11}^{(1)} x_1 + \theta_{12}^{(1)} x_2 + \theta_{13}^{(1)} x_3)$$

$$a_2^{(2)} = g(\theta_{20}^{(1)} x_0 + \theta_{21}^{(1)} x_1 + \theta_{22}^{(1)} x_2 + \theta_{23}^{(1)} x_3)$$

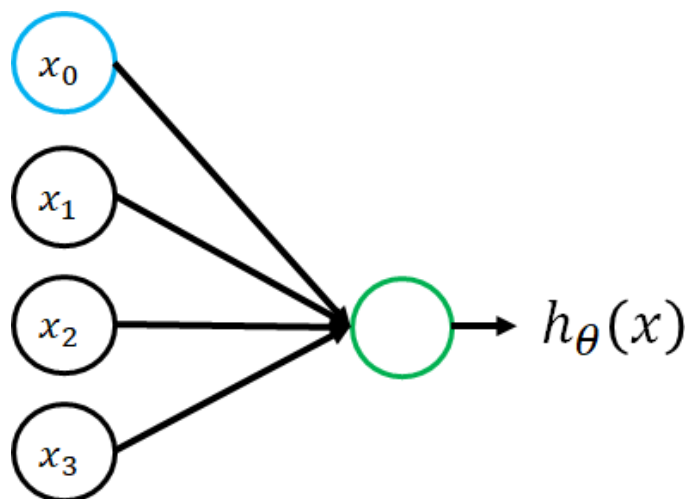
$$a_3^{(2)} = g(\theta_{30}^{(1)} x_0 + \theta_{31}^{(1)} x_1 + \theta_{32}^{(1)} x_2 + \theta_{33}^{(1)} x_3)$$

$$h_{\theta}(x) = a_1^{(3)} = g(\theta_{10}^{(2)} a_1^{(2)} + \theta_{11}^{(2)} a_2^{(2)} + \theta_{12}^{(2)} a_3^{(2)} + \theta_{13}^{(2)} a_4^{(2)})$$

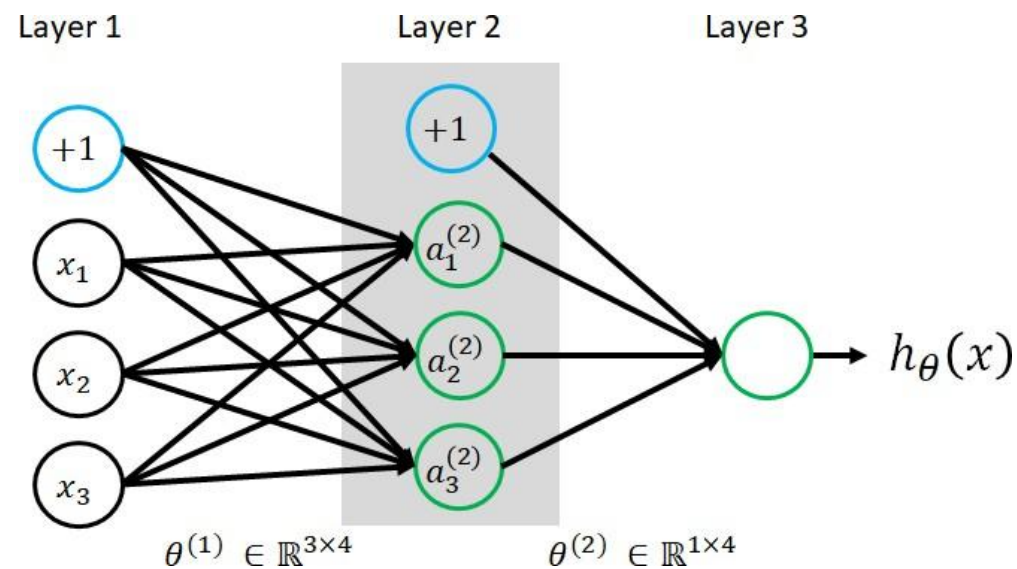
- Notation used in computation:
 $a_i^{(j)}$: activation of neuron i in layer j
 θ^j : matrix of weights make up the mapping function from layer j to $j + 1$
 s_j : number of neurons in layer j not including bias
- Note that, if the network has s_j neurons in layer j and s_{j+1} neurons in layer $j + 1$, then, the matrix dimension for θ^j is $s_{j+1} \times (s_j + 1)$

Feature learning capability

Logistic regression

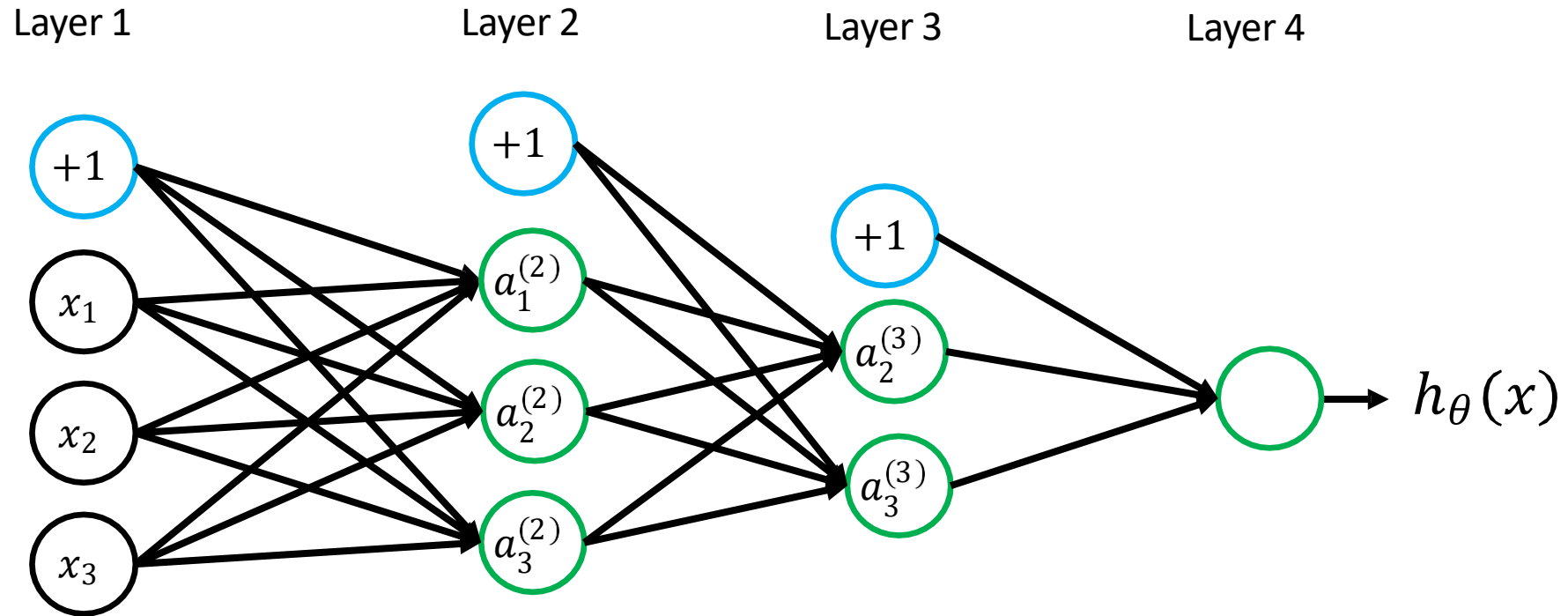


Neural Networks



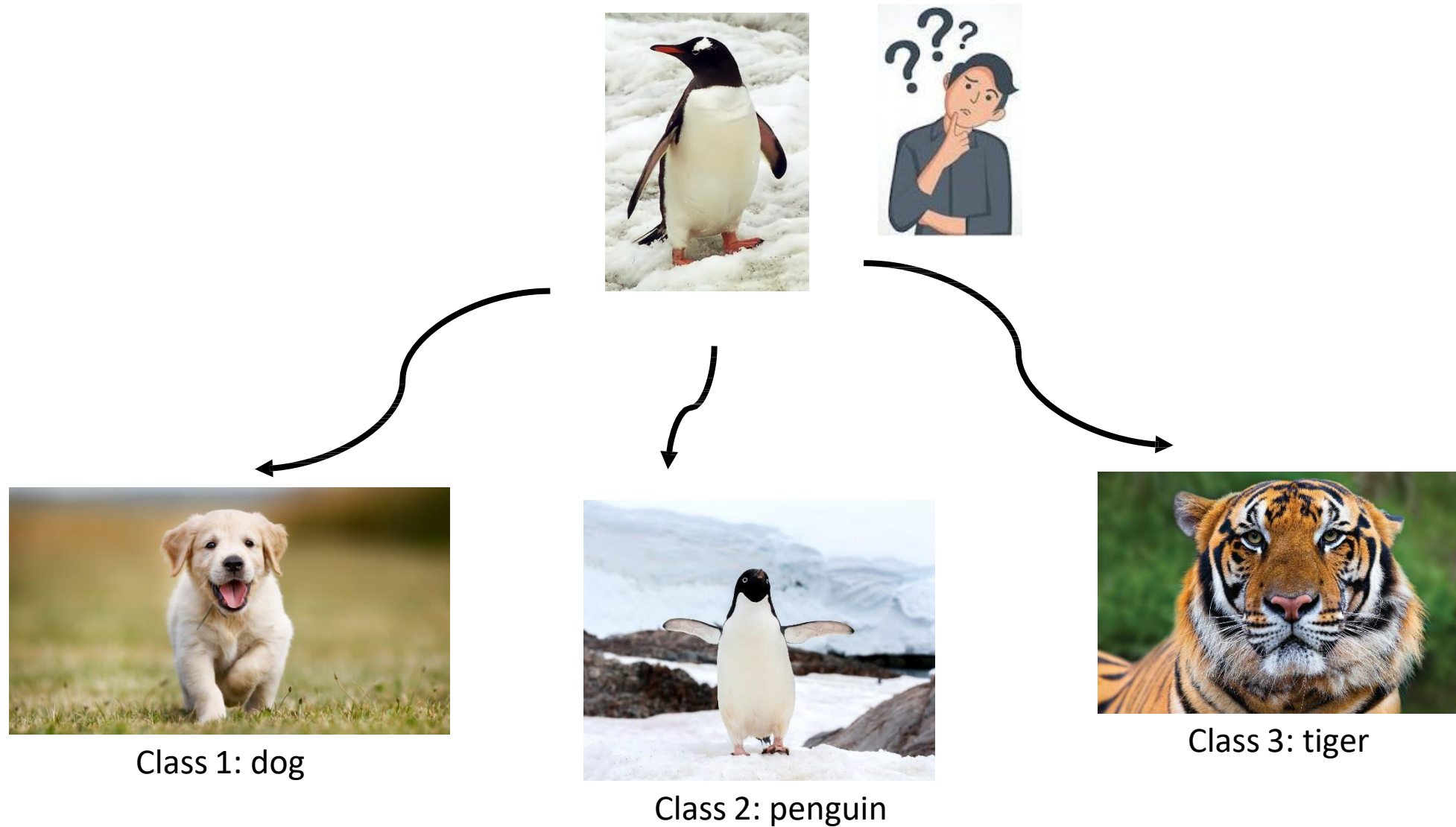
- Logistic regression can be considered as a neural network without a hidden layer. Its mapping function connects the input x to the output y by a sigmoid function of a linear combination of features.
- A neural network is **more complex** than logistic regression. Normally, it has at least one hidden layer, which consists of the new features ($a^{(2)}$) that are learned as a function of $f(\theta^{(1)}, x)$.
 - With such flexibility in feature learning, neural networks can learn more complex features and output a more complex non-linear hypothesis.

Neural network with deeper layers

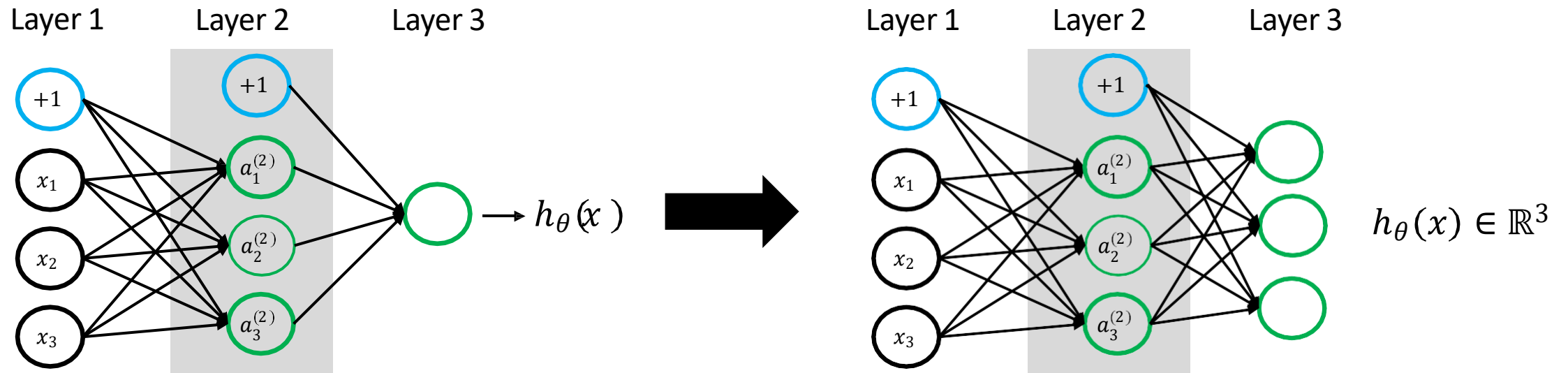


The complexity of the features increases

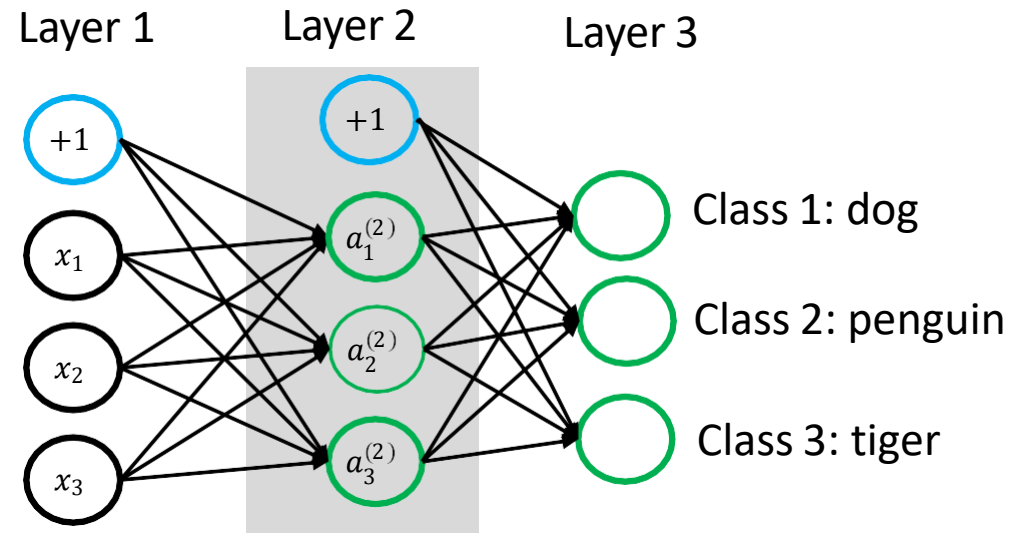
Multiclass classification



Multiple output neurons



Multiple output neurons



- Given N number of data points (training set) $= (x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(N)}, y^{(N)})$
- We no longer label $y^{(n)} \in \{1, 2, 3\}$ but $y^{(n)} \in \mathbb{R}^3$ as follows:



Class 1: dog

$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$



Class 2: penguin

$$y = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$



Class 3: tiger

$$y = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Multiple output neurons



$$y = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$

Class 1: dog



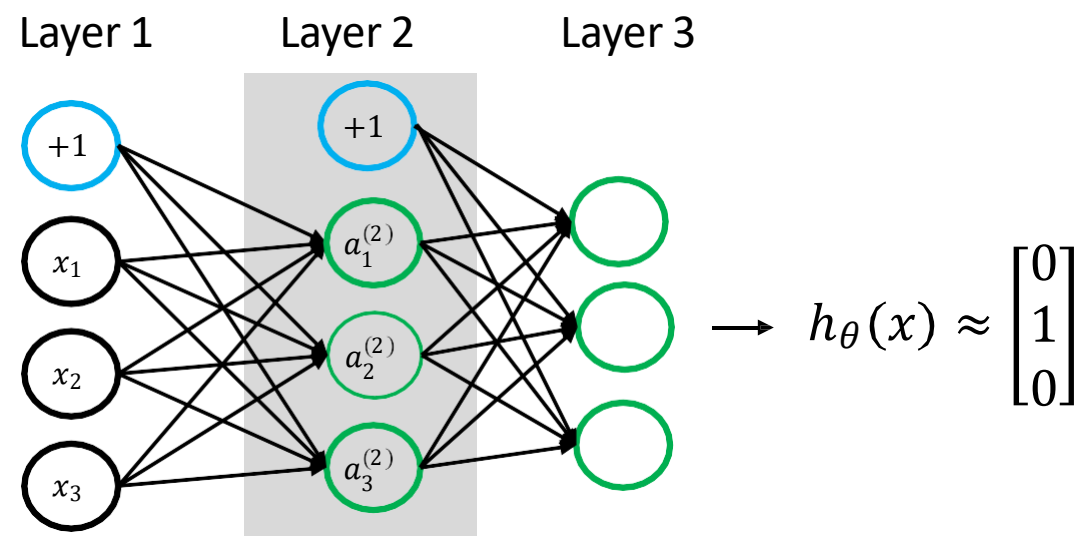
$$y = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Class 2: penguin

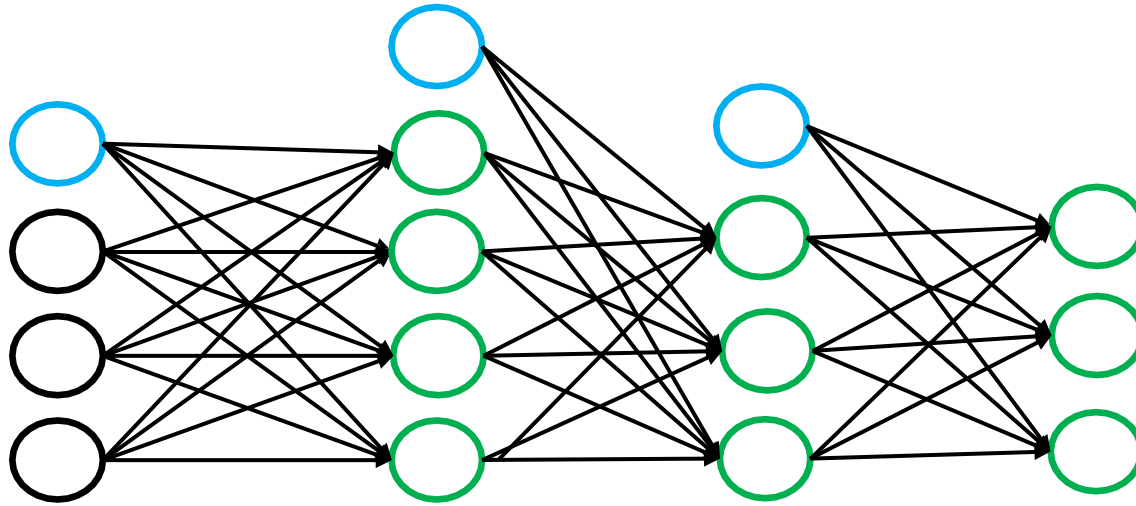


$$y = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Class 3: tiger



Cost function (Classification)



$$K = 3$$

$$L = 4$$

$$s_1 = 3 \quad s_2 = 4 \quad s_3 = 3 \quad s_L = 3$$

$$y \in \mathbb{R}^3$$

$$h_{\theta}(x) \in \mathbb{R}^3$$

- Terminology used:

K : number of output neurons

L : total number of layers in network

s_l : Number of neurons in layer l (not including bias)

- For multiclass classification (K classes where $K \geq 3$)

$$y \in \mathbb{R}^K$$

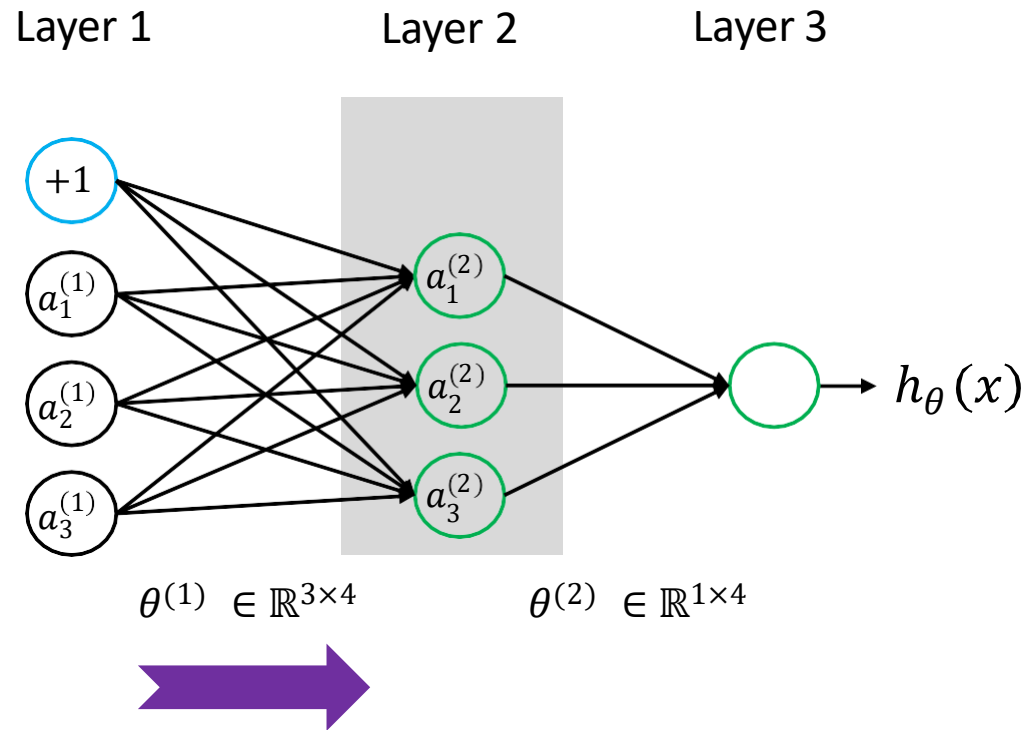
$$h_{\theta}(x) \in \mathbb{R}^K$$

- For binary classification:

$$y \in \{0,1\}$$

$$h_{\theta}(x) \in \mathbb{R}$$

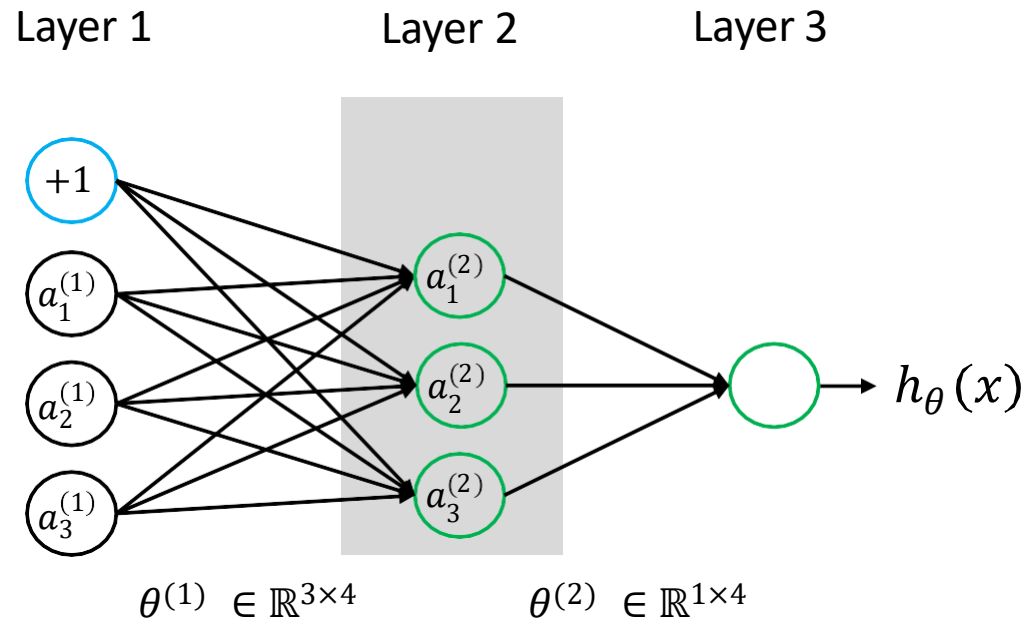
Forward propagation



$$z^{(2)} = \theta^{(1)} a^{(1)}$$

$$\begin{aligned} a_1^{(2)} &= g(\theta_{10}^{(1)} a_0^{(1)} + \theta_{11}^{(1)} a_1^{(1)} + \theta_{12}^{(1)} a_2^{(1)} + \theta_{13}^{(1)} a_3^{(1)}) \\ a_2^{(2)} &= g(\theta_{20}^{(1)} a_0^{(1)} + \theta_{21}^{(1)} a_1^{(1)} + \theta_{22}^{(1)} a_2^{(1)} + \theta_{23}^{(1)} a_3^{(1)}) \\ a_3^{(2)} &= g(\theta_{30}^{(1)} a_0^{(1)} + \theta_{31}^{(1)} a_1^{(1)} + \theta_{32}^{(1)} a_2^{(1)} + \theta_{33}^{(1)} a_3^{(1)}) \end{aligned}$$

Forward propagation

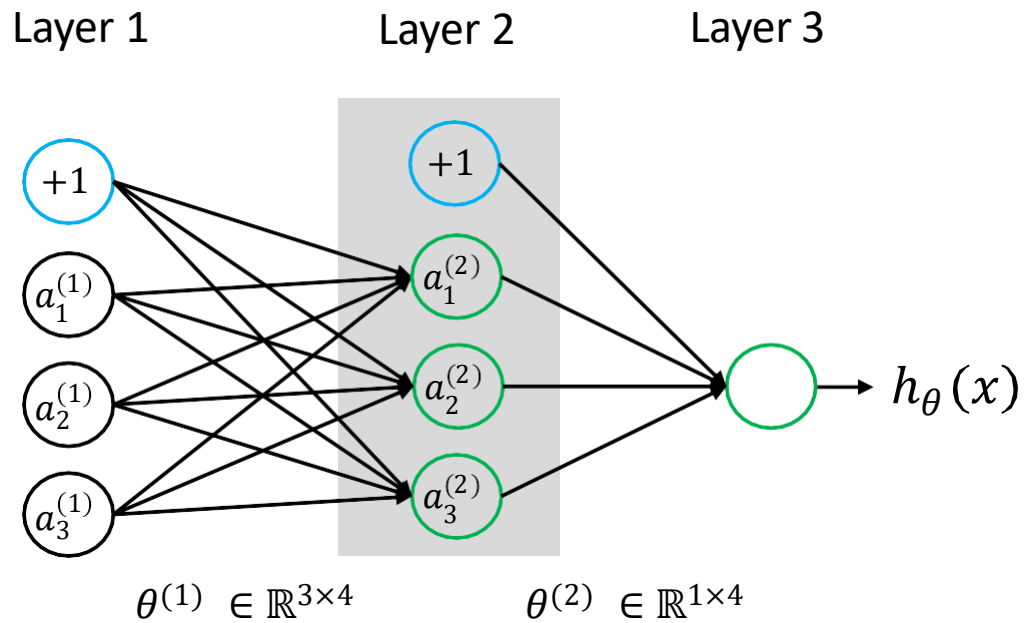


$$z^{(2)} = \theta^{(1)} a^{(1)}$$

$$a^{(2)} = g(z^{(2)})$$

$$\begin{aligned} a_1^{(2)} &= g(\theta_{10}^{(1)} a_0^{(1)} + \theta_{11}^{(1)} a_1^{(1)} + \theta_{12}^{(1)} a_2^{(1)} + \theta_{13}^{(1)} a_3^{(1)}) \\ a_2^{(2)} &= g(\theta_{20}^{(1)} a_0^{(1)} + \theta_{21}^{(1)} a_1^{(1)} + \theta_{22}^{(1)} a_2^{(1)} + \theta_{23}^{(1)} a_3^{(1)}) \\ a_3^{(2)} &= g(\theta_{30}^{(1)} a_0^{(1)} + \theta_{31}^{(1)} a_1^{(1)} + \theta_{32}^{(1)} a_2^{(1)} + \theta_{33}^{(1)} a_3^{(1)}) \end{aligned}$$

Forward propagation



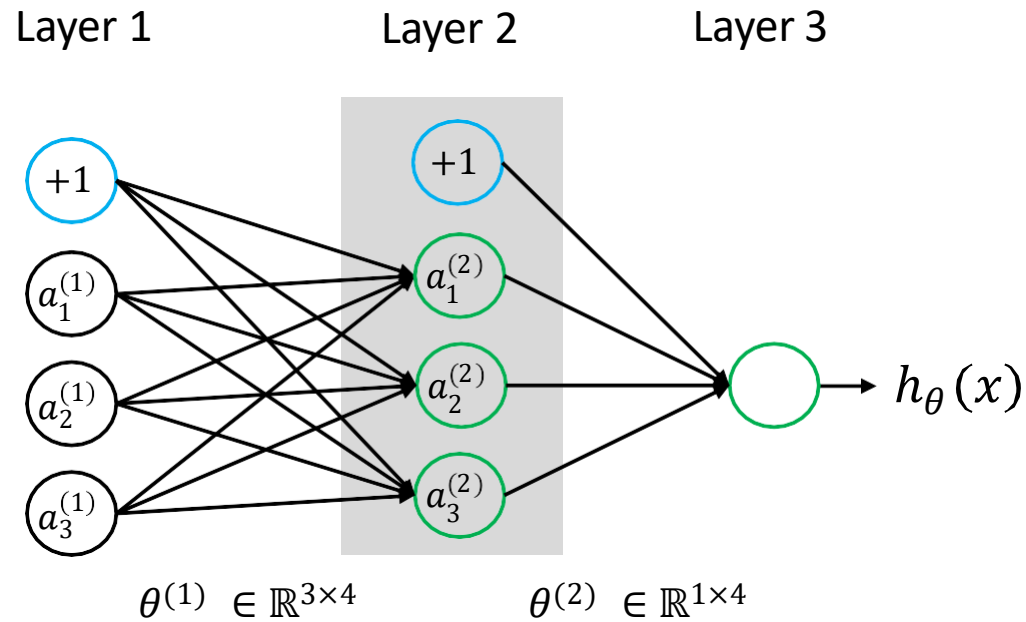
$$z^{(2)} = \theta^{(1)} a^{(1)}$$

$$a^{(2)} = g(z^{(2)})$$

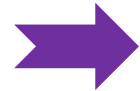
$$z^{(3)} = \theta^{(2)} a^{(2)}$$

$$\begin{aligned} a_1^{(2)} &= g(\theta_{10}^{(1)} a_0^{(1)} + \theta_{11}^{(1)} a_1^{(1)} + \theta_{12}^{(1)} a_2^{(1)} + \theta_{13}^{(1)} a_3^{(1)}) \\ a_2^{(2)} &= g(\theta_{20}^{(1)} a_0^{(1)} + \theta_{21}^{(1)} a_1^{(1)} + \theta_{22}^{(1)} a_2^{(1)} + \theta_{23}^{(1)} a_3^{(1)}) \\ a_3^{(2)} &= g(\theta_{30}^{(1)} a_0^{(1)} + \theta_{31}^{(1)} a_1^{(1)} + \theta_{32}^{(1)} a_2^{(1)} + \theta_{33}^{(1)} a_3^{(1)}) \\ a_1^{(3)} &= g(\theta_{10}^{(2)} a_0^{(2)} + \theta_{11}^{(2)} a_1^{(2)} + \theta_{12}^{(2)} a_2^{(2)} + \theta_{13}^{(2)} a_3^{(2)}) \end{aligned}$$

Forward propagation

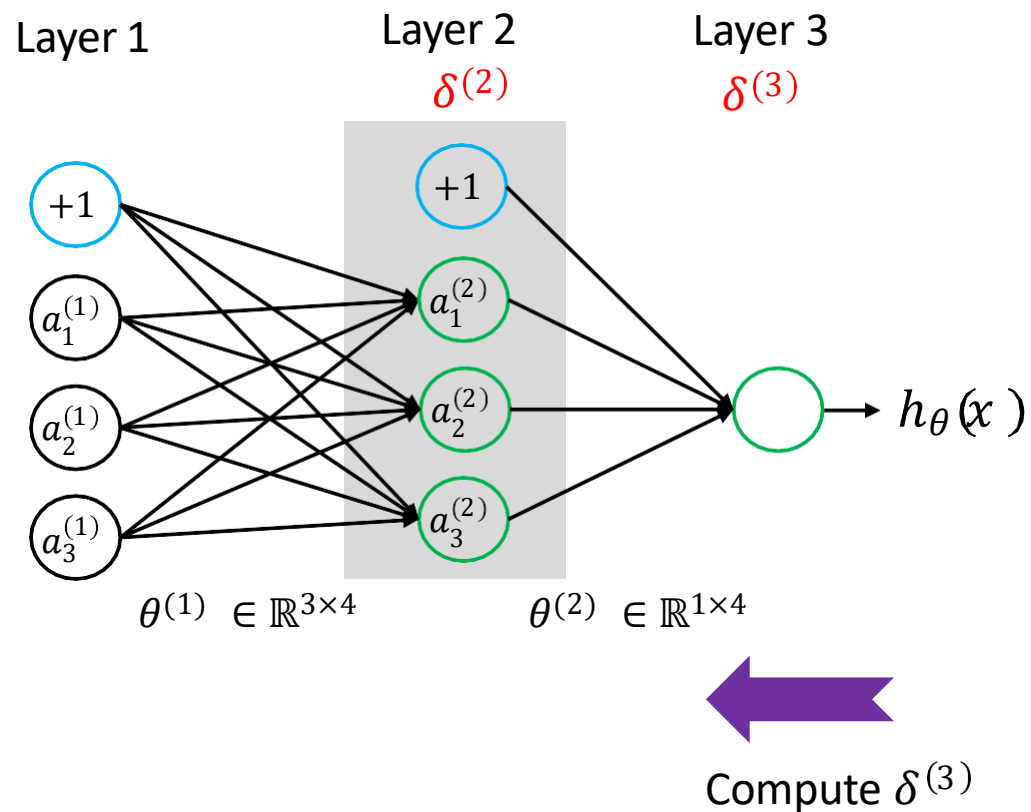


$$\begin{aligned} z^{(2)} &= \theta^{(1)} a^{(1)} \\ a^{(2)} &= g(z^{(2)}) \\ z^{(3)} &= \theta^{(2)} a^{(2)} \\ a^{(3)} &= g(z^{(3)}) \end{aligned}$$



$$\begin{aligned} a_1^{(2)} &= g(\theta_{10}^{(1)} a_0^{(1)} + \theta_{11}^{(1)} a_1^{(1)} + \theta_{12}^{(1)} a_2^{(1)} + \theta_{13}^{(1)} a_3^{(1)}) \\ a_2^{(2)} &= g(\theta_{20}^{(1)} a_0^{(1)} + \theta_{21}^{(1)} a_1^{(1)} + \theta_{22}^{(1)} a_2^{(1)} + \theta_{23}^{(1)} a_3^{(1)}) \\ a_3^{(2)} &= g(\theta_{30}^{(1)} a_0^{(1)} + \theta_{31}^{(1)} a_1^{(1)} + \theta_{32}^{(1)} a_2^{(1)} + \theta_{33}^{(1)} a_3^{(1)}) \\ a_1^{(3)} &= g(\theta_{10}^{(2)} a_0^{(2)} + \theta_{11}^{(2)} a_1^{(2)} + \theta_{12}^{(2)} a_2^{(2)} + \theta_{13}^{(2)} a_3^{(2)}) \end{aligned}$$

Backward propagation



$\delta_j^{(l)}$ = error of j -th neuron in layer l

$$z^{(2)} = \theta^{(1)} a^{(1)}$$

$$a^{(2)} = g(z^{(2)})$$

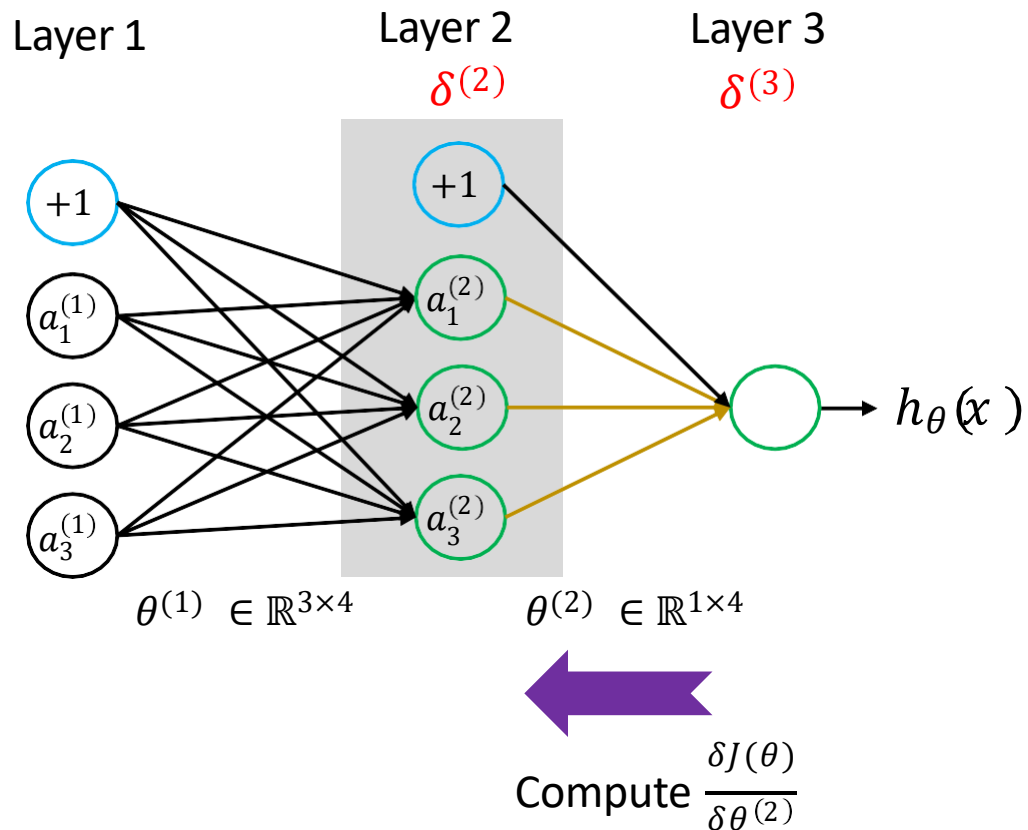
$$z^{(3)} = \theta^{(2)} a^{(2)}$$

$$a^{(3)} = g(z^{(3)})$$

$$h_{\theta}(x) = a^{(3)}$$

$$\delta^{(3)} = \frac{\delta J(\theta)}{\delta z^{(3)}} = \frac{\delta J(\theta)}{\delta a^{(3)}} \frac{\delta a^{(3)}}{\delta z^{(3)}} = (a^{(3)} - y) g'(z^{(3)})$$

Backward propagation



$\delta_j^{(l)}$ = error of j -th neuron in layer l

$$z^{(2)} = \theta^{(1)} a^{(1)}$$

$$a^{(2)} = g(z^{(2)})$$

$$z^{(3)} = \theta^{(2)} a^{(2)}$$

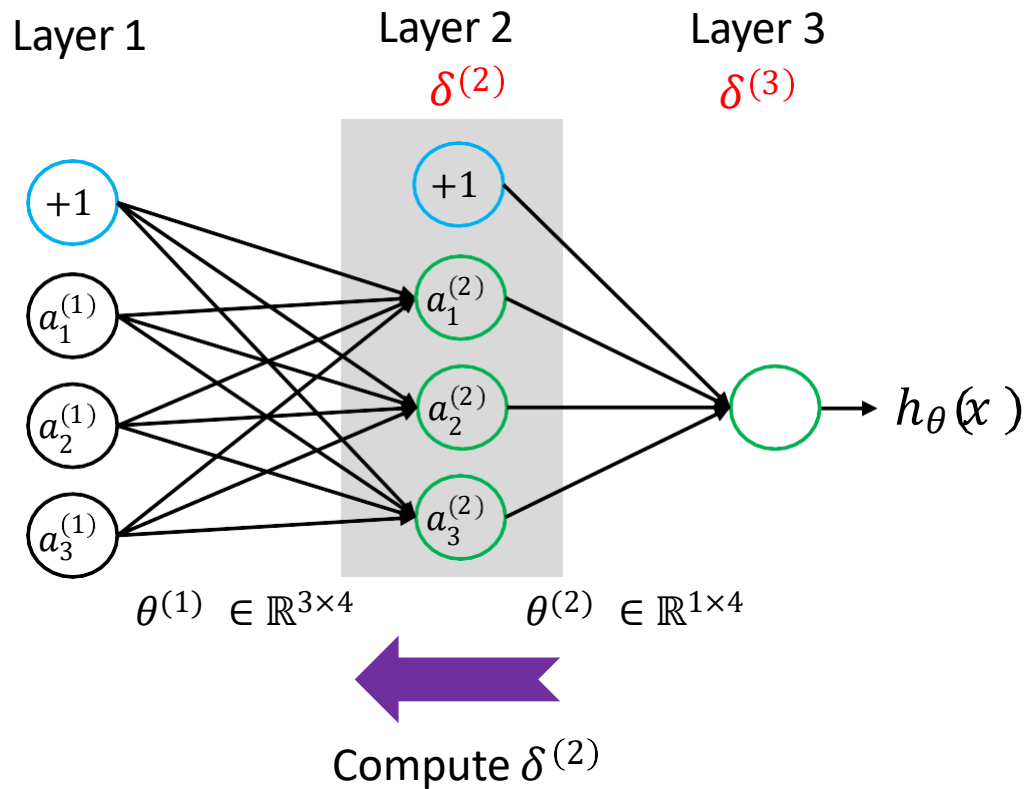
$$a^{(3)} = g(z^{(3)})$$

$$h_{\theta}(x) = a^{(3)}$$

$$\delta^{(3)} = \frac{\delta J(\theta)}{\delta z^{(3)}} = \frac{\delta J(\theta) \delta a^{(3)}}{\delta a^{(3)} \delta z^{(3)}} = (a^{(3)} - y) g'(z^{(3)})$$

$$\frac{\delta J(\theta)}{\delta \theta^{(2)}} = \frac{\delta J(\theta) \delta z^{(3)}}{\delta z^{(3)} \delta \theta^{(2)}} = \delta^{(3)} a^{(2)}$$

Backward propagation



$\delta_j^{(l)}$ = error of j -th neuron in layer l

$$z^{(2)} = \theta^{(1)} a^{(1)}$$

$$a^{(2)} = g(z^{(2)})$$

$$z^{(3)} = \theta^{(2)} a^{(2)}$$

$$a^{(3)} = g(z^{(3)})$$

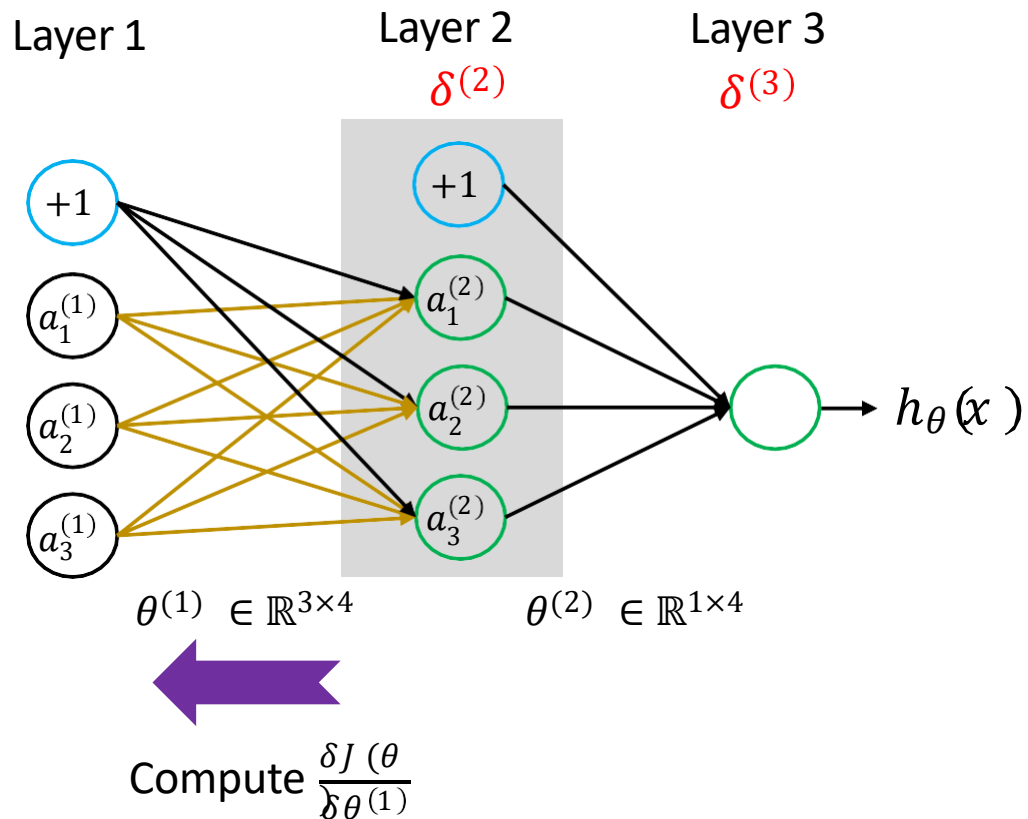
$$h_{\theta}(x) = a^{(3)}$$

$$\delta^{(3)} = \frac{\delta J(\theta)}{\delta z^{(3)}} = \frac{\delta J(\theta) \delta a^{(3)}}{\delta a^{(3)} \delta z^{(3)}} = (a^{(3)} - y) g'(z^{(3)})$$

$$\frac{\delta J(\theta)}{\delta \theta^{(2)}} = \frac{\delta J(\theta) \delta z^{(3)}}{\delta z^{(3)} \delta \theta^{(2)}} = \delta^{(3)} a^{(2)}$$

$$\begin{aligned} \delta^{(2)} &= \frac{\delta J(\theta)}{\delta z^{(2)}} = \frac{\delta J(\theta) \delta z^{(3)} \delta a^{(2)}}{\delta z^{(3)} \delta a^{(2)} \delta z^{(2)}} \\ &= \delta^{(3)} (\theta^{(2)})' g'(z^{(2)}) \\ &= (\theta^{(2)})^{(T)} \delta^{(3)} g'(z^{(2)}) \end{aligned}$$

Backward propagation



$\delta_j^{(l)}$ = error of j -th neuron in layer l

$$z^{(2)} = \theta^{(1)} a^{(1)}$$

$$a^{(2)} = g(z^{(2)})$$

$$z^{(3)} = \theta^{(2)} a^{(2)}$$

$$a^{(3)} = g(z^{(3)})$$

$$h_{\theta}(x) = a^{(3)}$$

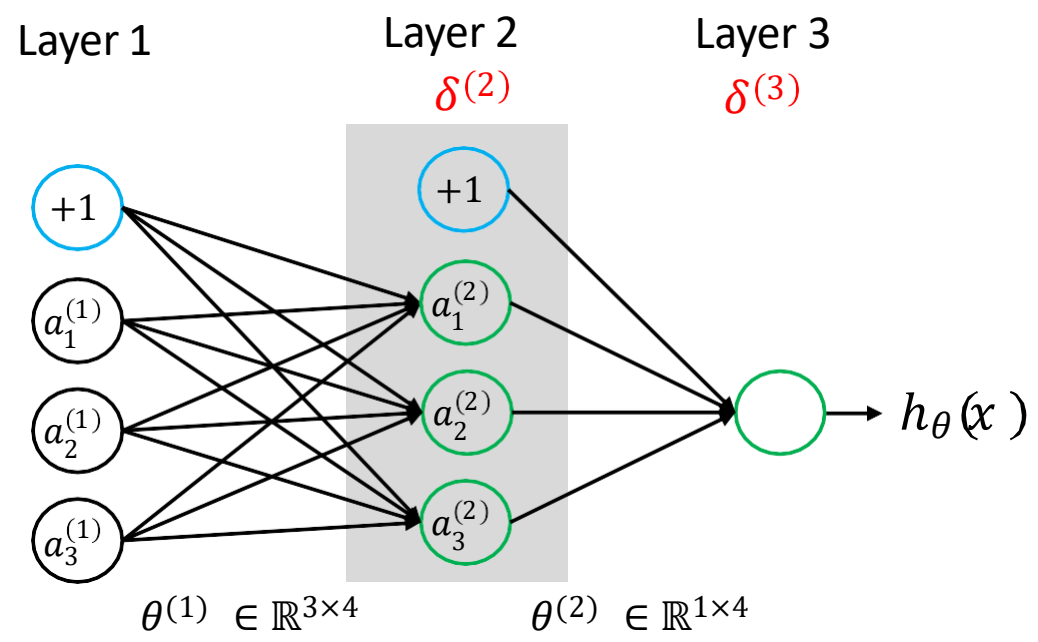
$$\delta^{(3)} = \frac{\delta J(\theta)}{\delta z^{(3)}} = \frac{\delta J(\theta) \delta a^{(3)}}{\delta a^{(3)} \delta z^{(3)}} = (a^{(3)} - y) g'(z^{(3)})$$

$$\frac{\delta J(\theta)}{\delta \theta^{(2)}} = \frac{\delta J(\theta) \delta z^{(3)}}{\delta z^{(3)} \delta \theta^{(2)}} = \delta^{(3)} a^{(2)}$$

$$\begin{aligned} \delta^{(2)} &= \frac{\delta J(\theta)}{\delta z^{(2)}} = \frac{\delta J(\theta) \delta z^{(3)} \delta a^{(2)}}{\delta z^{(3)} \delta a^{(2)} \delta z^{(2)}} \\ &= \delta^{(3)} (\theta^{(2)})^T g'(z^{(2)}) \\ &= (\theta^{(2)})^T \delta^{(3)} \odot g'(z^{(2)}) \end{aligned}$$

$$\frac{\delta J(\theta)}{\delta \theta^{(1)}} = \frac{\delta J(\theta) \delta z^{(3)} \delta a^{(2)} \delta z^{(2)}}{\delta z^{(3)} \delta a^{(2)} \delta z^{(2)} \delta \theta^{(1)}} = \delta^{(2)} a^{(1)}$$

Backward propagation



Summary of equation of backpropagation:

$$\delta^{(L)} = \frac{\delta J(\theta)}{\delta a^{(L)} \delta z^{(L)}}$$

$$\delta^{(l)} = (\theta^{(l)})^T \delta^{(l+1)} \odot g'(z^{(l)})$$

$$\frac{\delta J(\theta)}{\delta \theta_j^{(l)}} = \delta_j^{(l+1)} a_k^{(l)}$$

$$\frac{\delta J(\theta)}{\delta \theta_j^{(l)}} = \delta_j^{(l+1)}$$

$\delta_j^{(l)}$ = error of j -th neuron in layer l

$$z^{(2)} = \theta^{(1)} a^{(1)}$$

$$a^{(2)} = g(z^{(2)})$$

$$z^{(3)} = \theta^{(2)} a^{(2)}$$

$$a^{(3)} = g(z^{(3)})$$

$$h_{\theta}(x) = a^{(3)}$$

$$\delta^{(3)} = \frac{\delta J(\theta)}{\delta z^{(3)}} = \frac{\delta J(\theta) \delta a^{(3)}}{\delta a^{(3)} \delta z^{(3)}} = (a^{(3)} - y) g'(z^{(3)})$$

$$\frac{\delta J(\theta)}{\delta \theta^{(2)}} = \frac{\delta J(\theta) \delta z^{(3)}}{\delta z^{(3)} \delta \theta^{(2)}} = \delta^{(3)} a^{(2)}$$

$$\delta^{(2)} = \frac{\delta J(\theta)}{\delta z^{(2)}} = \frac{\delta J(\theta) \delta z^{(3)} \delta a^{(2)}}{\delta z^{(3)} \delta a^{(2)} \delta z^{(2)}} = \delta^{(3)} (\theta^{(2)})^T g'(z^{(2)})$$

$$\frac{\delta J(\theta)}{\delta \theta^{(1)}} = \frac{\delta J(\theta) \delta z^{(3)} \delta a^{(2)} \delta z^{(2)}}{\delta z^{(3)} \delta a^{(2)} \delta z^{(2)} \delta \theta^{(1)}} = \delta^{(2)} a^{(1)}$$

Gradients are computed using chain rule of differentiation

No $\delta^{(1)}$!

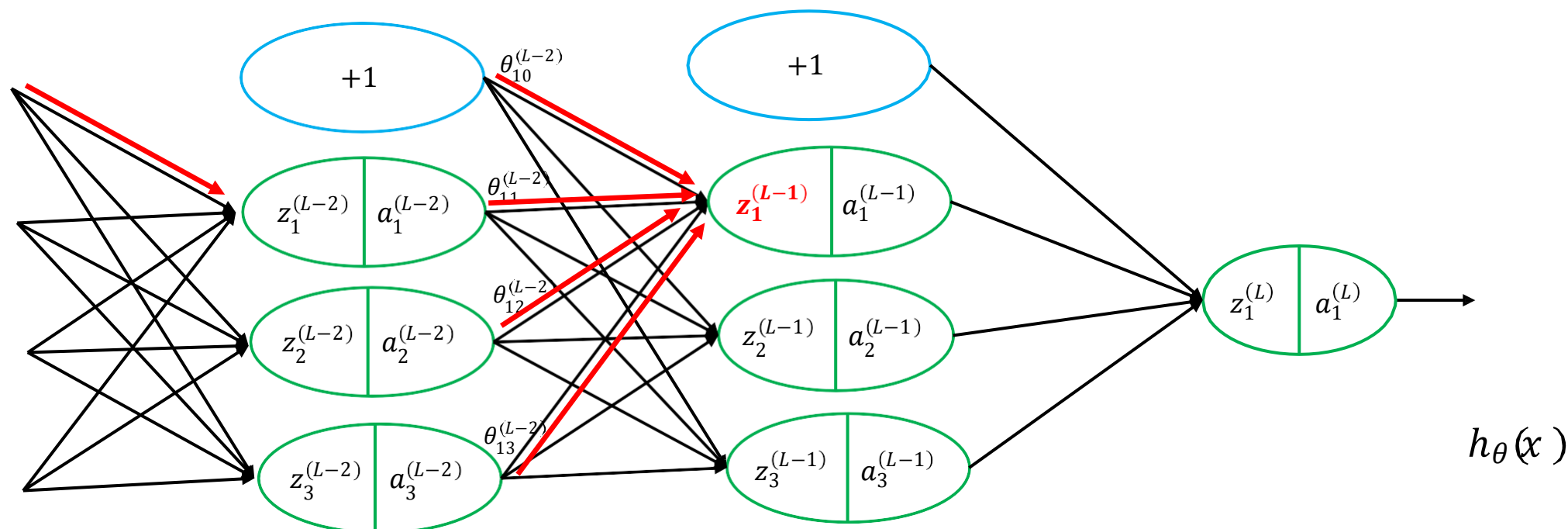
Backpropagation algorithm

1. **Input:** given N number of data points (training set) = $(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(N)}, y^{(N)})$
2. **Feedforward:** For each $l = 1, 2, \dots, L$, compute $z^{(l+1)} = \theta^{(l)} a^{(l)}$ and $a^{(l+1)} = g(z^{(l+1)})$
3. **Output error:** Compute $\delta^{(L)}$
4. **Backpropagate the error:** For each $l = 2, \dots, L$, compute $\delta^{(l)}$
5. **Gradient computation:** Compute the gradients for the weights and biases
6. **Optimization algorithm:** You can use any gradient descent optimization algorithm to minimize the error function $J(\theta)$.

Backpropagation intuition

- Forward propagation simply computes the weighted sum of $z^{(l)}$ from the left to the right.

$$\text{E.g. } z^{(L-1)} = \theta_{10}^{(L-2)} a_0^{(L-2)} + \theta_{11}^{(L-2)} a_1^{(L-2)} + \theta_{12}^{(L-2)} a_2^{(L-2)} + \theta_{13}^{(L-2)} a_3^{(L-2)}$$

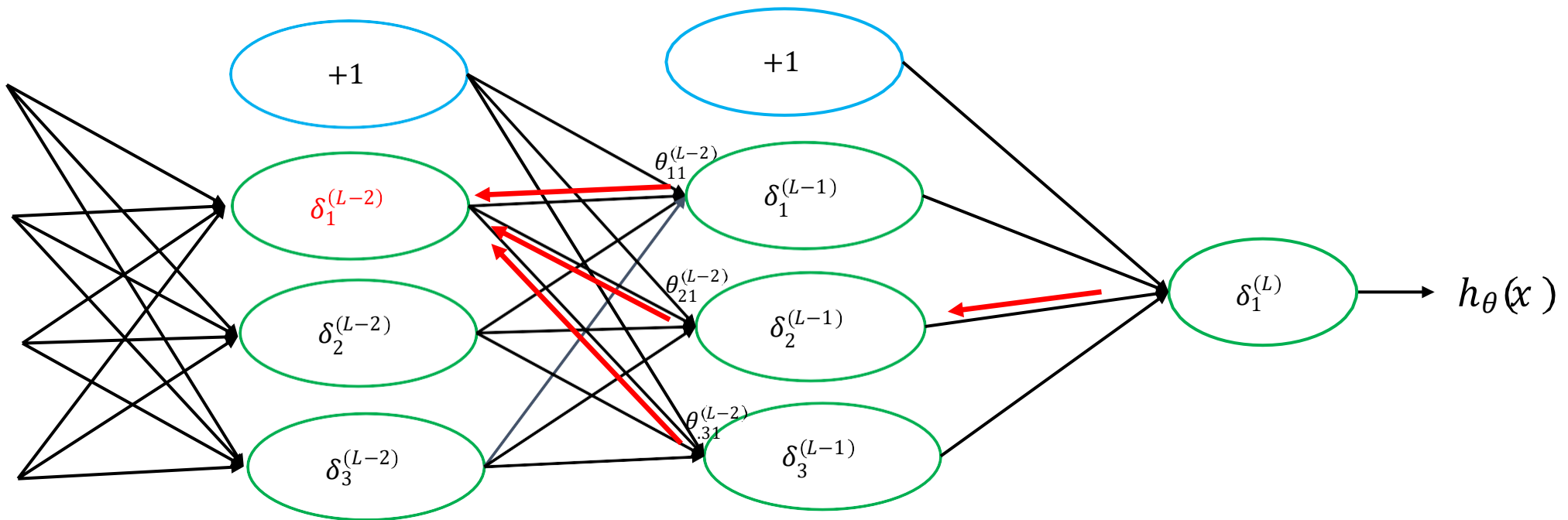


Backpropagation intuition

- Backward propagation computes the error term $\delta^{(l)}$ from the right to the left.

$$\text{E.g. } \delta_1^{(L-2)} = \theta_{11}^{(L-2)} \delta_1^{(L-1)} + \theta_{21}^{(L-2)} \delta_2^{(L-1)} + \theta_{31}^{(L-2)} \delta_3^{(L-1)}$$

- Measure of how much will we like to modify the weights $\theta_{jk}^{(l)}$ in order to affect this intermediate value of the calculation so as to influence the final result $h_\theta(x)$ and the cost function $J(\theta)$.



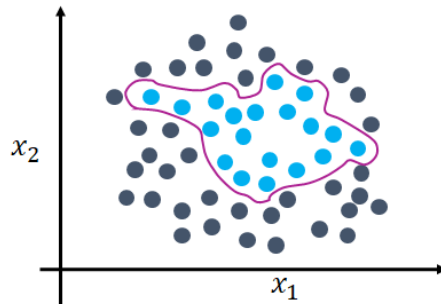
Random Initialization of Weights

- At the start of training, neural network weights are initialized with random values.
- Random initialization helps neurons learn different features.

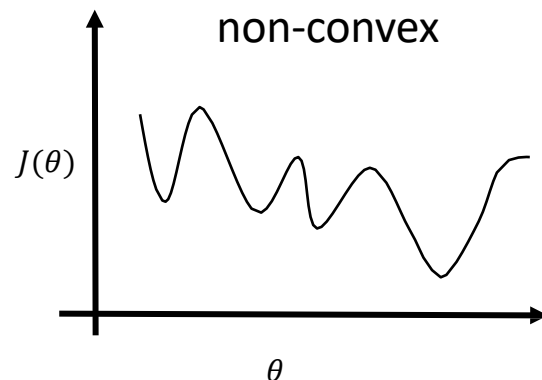
LR vs ANN

Neural Network

- Neural networks are much better for a **complex nonlinear hypothesis**.

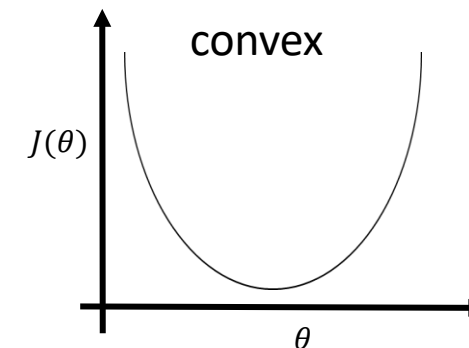


- A neural network is in general **not convex**. It can suffer from multiple local minima.



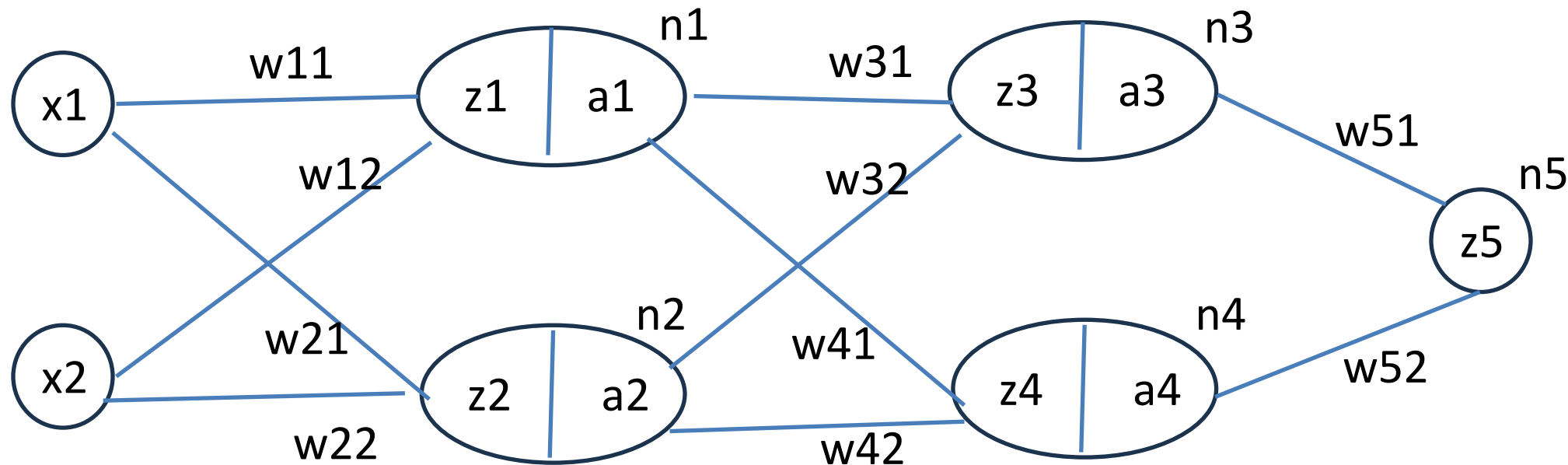
Logistic regression

- LR is useful for dataset where the classes are more or less “linearly separable”
- For “relatively” very **small** dataset.
- It is a great, robust model for **simple** classification tasks
- LR cost function (or max-entropy) is **convex**, and thus we are guaranteed to find the global cost minimum.



Let's walk through an example.

Forward propagation



$$z1 = w11 * x1 + w12 * x2 + b1$$
$$a1 = g(z1)$$

$$z3 = w31 * a1 + w32 * a2 + b3$$
$$a3 = g(z3)$$

$$z5 = w51 * a3 + w52 * a4 + 5$$

z is the weighted sum (b is bias, shift the value)

g is the activation function that introduces nonlinearity. It controls how the weighted sum z is transformed into the neuron's output.

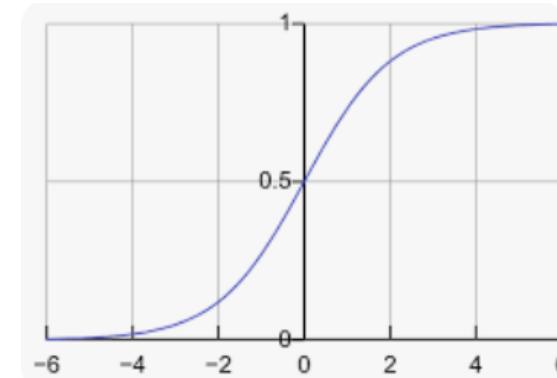
Activation Function

Activation function introduces nonlinearity. It controls how the weighted sum z is transformed into the neuron's output.

1. Sigmoid

Output range: 0 to 1, it is a smooth S-shaped curve

$$g(z) = \frac{1}{1 + e^{-z}}$$



2. ReLU (Rectified Linear Unit)

Output: 0 if $z \leq 0$ otherwise z

$$g(z) = \max(0, z)$$



Loss Function / Cost Function

1. Mean Squared Error (MSE)

Used for regression problems.

$$J = \frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2$$

where

- y_i = true value
- $\hat{y}_i$ = predicted value
- n = number of samples

2. Cross-Entropy Loss (Binary)

Used for **binary** classification.

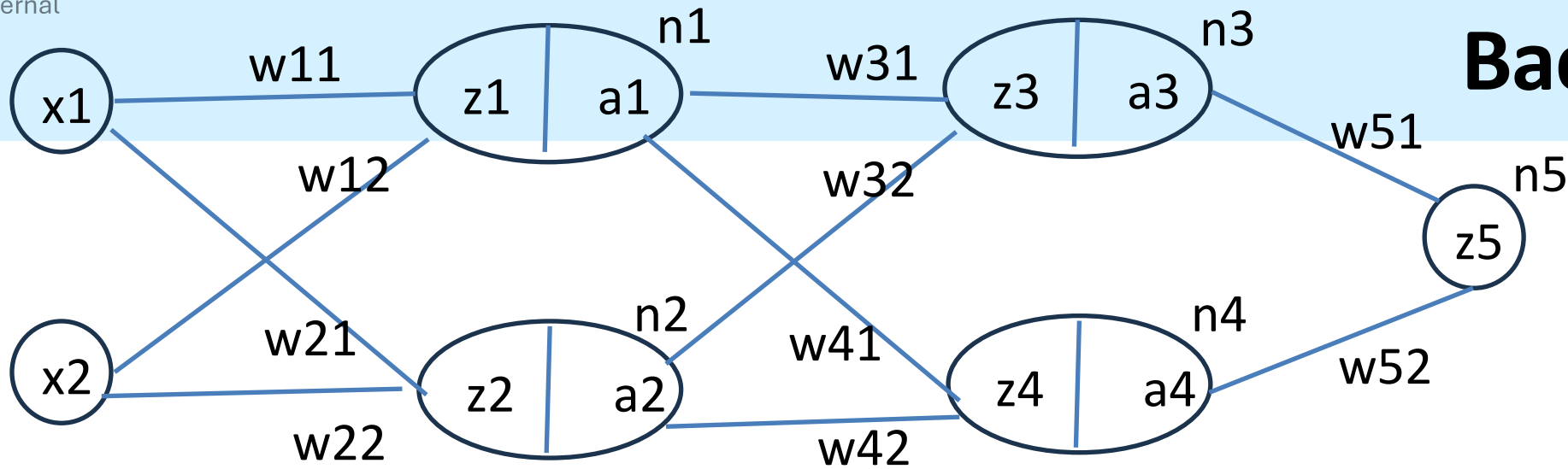
$$J = -\frac{1}{n} \sum_{i=1}^n [y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)]$$

where

- $y_i \in \{0, 1\}$
- $\hat{y}_i$ = predicted probability

Idea: penalizes wrong probability predictions.

Back propagation



Assume we use a loss function to compare the actual value and the predicted value, producing the loss L . How will a small change in w_{51} affect the loss L ?

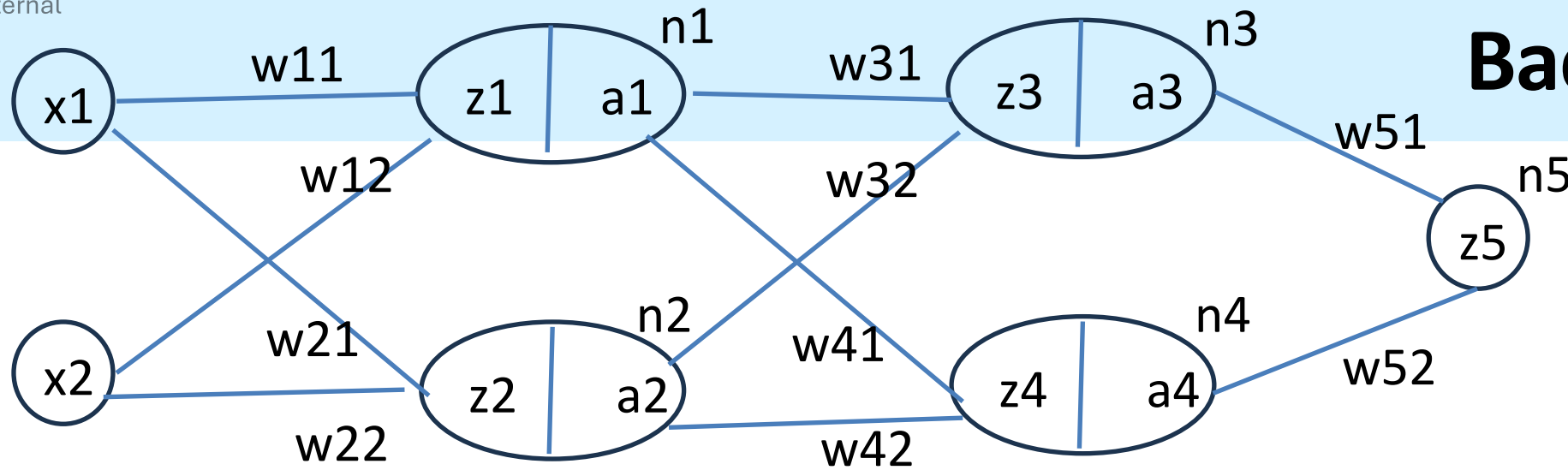
$$\frac{\partial L}{\partial w_{51}} = \frac{\partial L}{\partial z_5} \cdot \frac{\partial z_5}{\partial w_{51}} \quad \text{use the chain rule, } w_{51} \text{ affects } z_5, z_5 \text{ influences } L$$

Define $\delta_5 = \frac{\partial L}{\partial z_5}$ and $\frac{\partial z_5}{\partial w_{51}} = a_3$ so $\frac{\partial L}{\partial w_{51}} = \delta_5 a_3$

The update of w_{51} is proportional to how much w_{51} influences the loss L .
Large influence $\rightarrow$ large update.

$$w_{51} \leftarrow w_{51} - \eta \frac{\partial L}{\partial w_{51}}$$

Back propagation



How will a small change in w_{31} affect the loss L ?

$$w_{31} \rightarrow z_3 \rightarrow a_3 \rightarrow z_5 \rightarrow L$$

So

$$\frac{\partial L}{\partial w_{31}} = \frac{\partial L}{\partial z_5} \cdot \frac{\partial z_5}{\partial a_3} \cdot \frac{\partial a_3}{\partial z_3} \cdot \frac{\partial z_3}{\partial w_{31}}$$

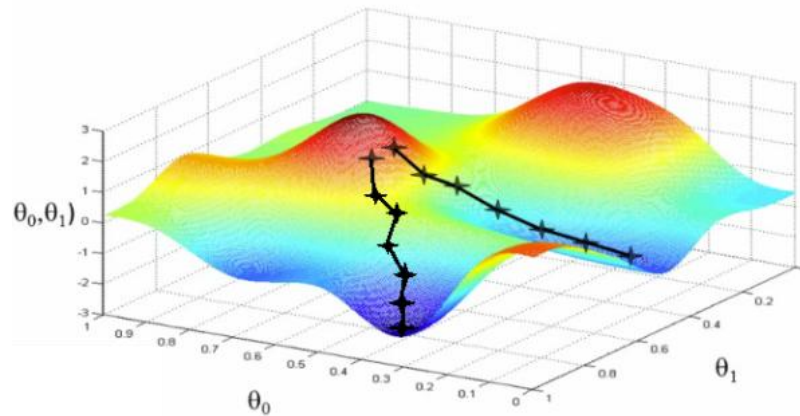
Define $\delta_3 = \frac{\partial L}{\partial z_3} = \delta_5 w_{51} g'(z_3)$ and $\frac{\partial z_3}{\partial w_{31}} = a_1$ so $\frac{\partial L}{\partial w_{31}} = \delta_3 a_1$

$$w_{31} \leftarrow w_{31} - \eta \frac{\partial L}{\partial w_{31}}$$

Through backpropagation, we can update all weights to reduce the error.

Gradient Descent

Gradient descent is an iterative algorithm that updates model parameters step by step in the direction that most reduces the error until the model reaches the minimum loss.



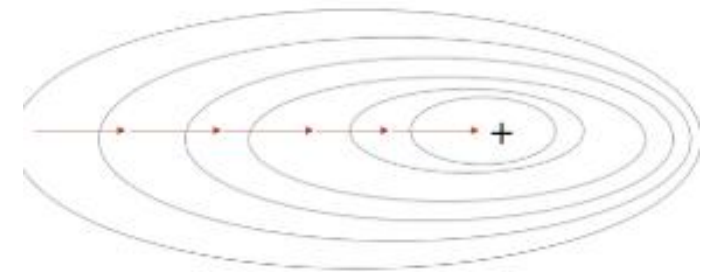
Types of Gradient Descent

Batch Gradient Descent – uses the entire dataset each update

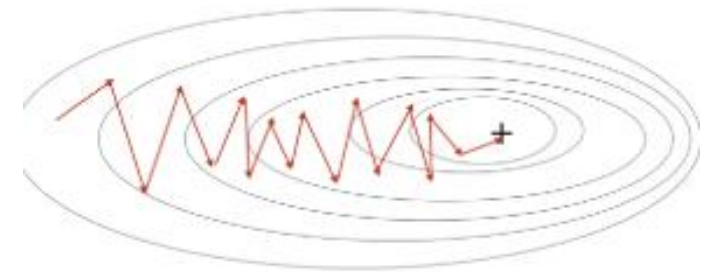
Stochastic Gradient Descent – uses one sample each update

Mini-Batch Gradient Descent – uses a small subset of samples

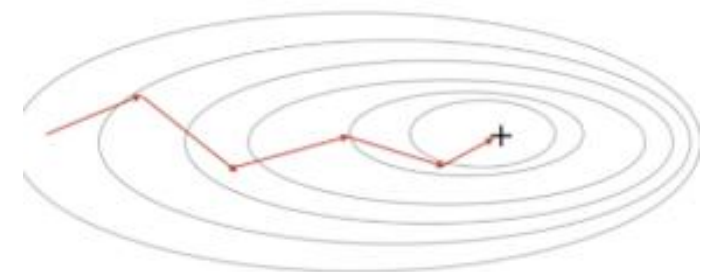
Gradient Descent



Stochastic Gradient Descent



Mini-Batch Gradient Descent



Neural Network Training Steps

Initialize parameters

Initialize all weights and biases with small random values.

Forward propagation

Pass the input through the network to produce a prediction.

Compute loss

Compare the predicted output with the true value to measure the error.

Backpropagation

Compute how each weight contributes to the error.

Update weights (Gradient Descent)

Adjust the weights to reduce the error.

Repeat

Repeat the process for many iterations until the error becomes small.

Next Lecture

❖ Convolutional neural network

